



Date :

Page No. :

Ch 4: Convolutional Neural Network

VI : Biological Inspiration : Visual Cortex

ConvNets are designed for visual tasks.

Computer Vision $\rightarrow$ In b/w ML/AI
and Images/~~and~~ videos.

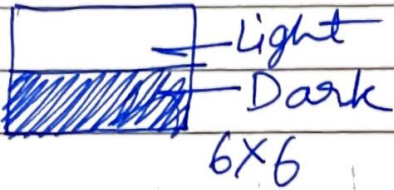
Hubel and Wiesel carried out an experiment.

Key findings were:-

- 1) There are some neurons in the visual cortex that fire when presented with lights at specific orientation/angles.
In Human brain
- 2) VI: Primary Visual Cortex.
 $\rightarrow$ Detects edges.
So, There are edge detector neurons, motion detectors, depth detectors, color detectors, shape detectors, etc.
- 3) Hierarchical structure of visual cortex neurons.

V2: Convolution: Edge Detection on Images

V1: Primary Visual Cortex: Edge Detector.
Grayscale Image.



Is there an edge in the image?
Horizontal
Sobel Edge Detector

zoomed

0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
255	255	255	255	255	255
255	255	255	255	255	255
255	255	255	255	255	255

1/p

6x6

*

+1	+2	+1
0	0	0
-1	-2	-1

=>

3x3

* Convolution operator

Kernel/
Filter/
Mask/
Operator

$$\begin{aligned} [n-k]+1 \\ [6-3]+1 \\ [3+1] \\ =4 \\ \therefore 4 \times 4 \\ \text{o/p} \end{aligned}$$

0	0	0	0
-1020	-1020	-1020	-1020
-1020	-1020	-1020	-1020
0	0	0	0

output

4x4

Normalize
using min & max



Date : _____

Page No. : _____

Convolution $\rightarrow$ Element wise multiplication on matrices and summing them up.

It is similar to dot product but on matrices and not on vectors

V3: Convolution: Padding and Strides

$$\begin{array}{ccc} 1 & n \times n & \xrightarrow[\text{kernel}]{k \times k} (n-k+1) \times (n-k+1) \\ \text{Image} & & \text{Feature Map} \end{array}$$

padding $\rightarrow$ 1 column on left & right each and 1 row on top and bottom each.

- 1) Zero padding
- 2) pad with value adjacent.

$$\begin{array}{ccc} 2 & n \times n & \xrightarrow[\text{kernel}]{k \times k} (n-k+2p+1) \times (n-k+2p+1) \\ & + \text{padding } p & \end{array}$$



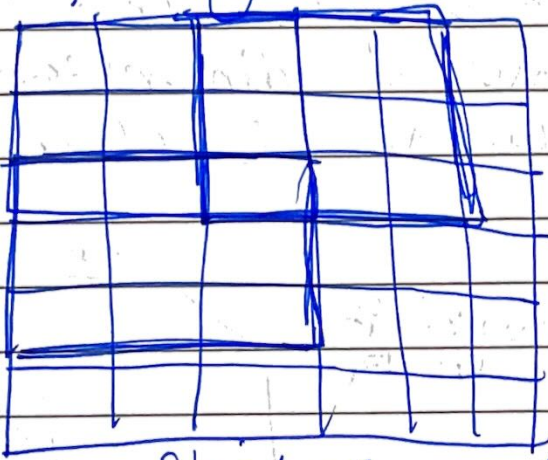
Date:

Page No.:

Stride

Shift by 1 was suggested by Sir!

Now we can even do, Shift by 2



$$n \times n \xrightarrow{\text{Stride}=s} \frac{(n-k)}{s} + 1 \times$$

$k \times k \text{ kernel}$

For image shrinkage

$$\left(\frac{(n-k)}{s} + 1 \right)$$

Feature Map

Where we apply both padding and stride:-

~~Complete Formula~~

$$\left(\frac{n-k+2p}{s} + 1 \right) \times \left(\frac{n-k+2p}{s} + 1 \right)$$

Feature Map

$$\frac{((p - \text{kernel}) + 2 \text{ padding})}{\text{Stride}} + 1$$



Date : _____

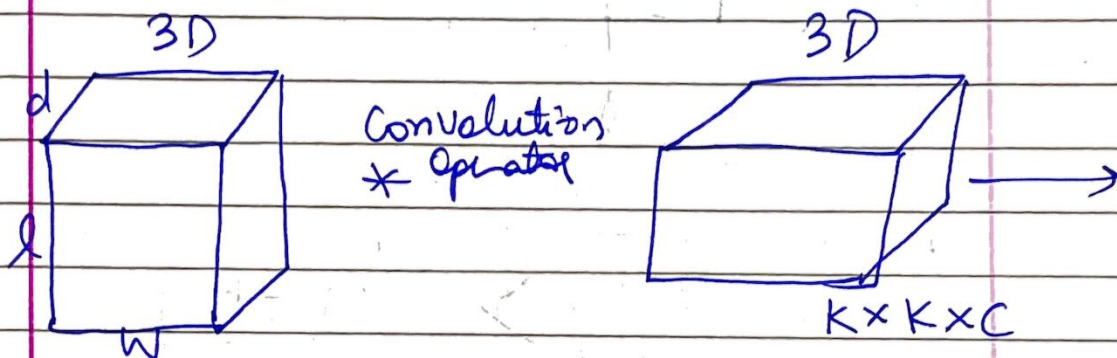
Page No. : _____

5/1

V4: Convolution over RGB images

Like in grayscale image matrix there are RGB image matrix with triplets of each Red, Green and blue values.

It can also be represented with a 3-D tensor of dimensions $n \times m \times C$ where $C = \text{channels}$.



$$(n - k + 1) \times (n - k + 1) \times$$

C of both the Matrices should be of same values. $C = \text{depth}$.

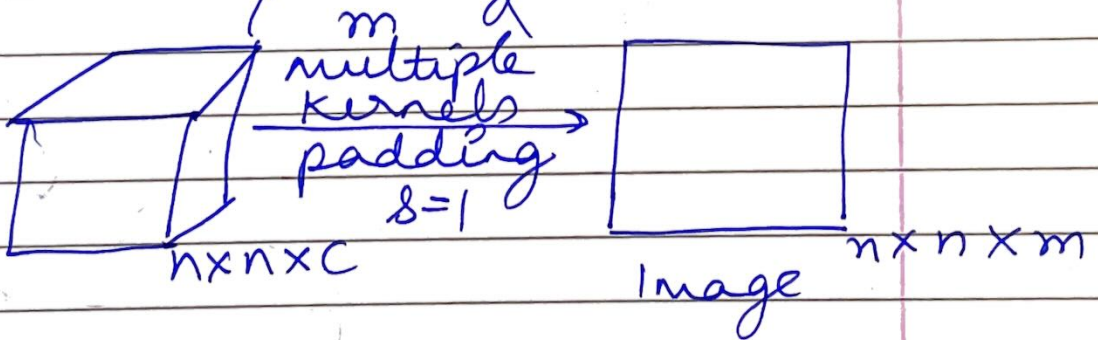
VS: Convolution layer

- 1) Biologically inspired from mammalian visual cortex.
- 2) multiple edge detectors from multiple kernels are obtained

CNN $\rightarrow$ learn kernel matrices.

$$\rightarrow I * M_{k \times k}$$

Different kernels can solve problem of different angled edges by using different kernel weights.



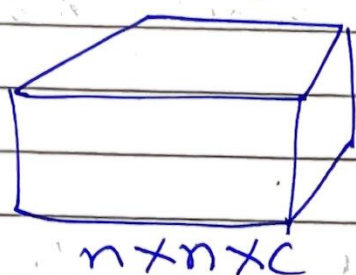
If I have 10 kernels I have 10 images which can be overlapped.

depth m depends on # of kernels.

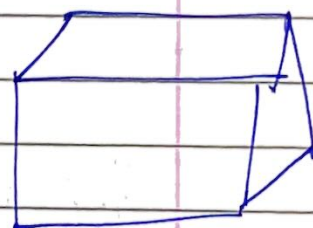
So hyperparameters $\Rightarrow$ # of kernels
padding
stride.

1) Input taken, convolve with kernel, with sufficient padding and stride

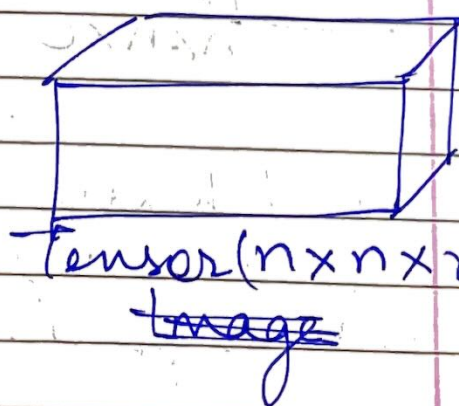
2) Take the output of above and perform element wise ReLU you get a final matrix



$m(K \times K \times c)$
convolution kernels
padding
stride



$n \times n \times m$
element
wise
ReLU





Date :

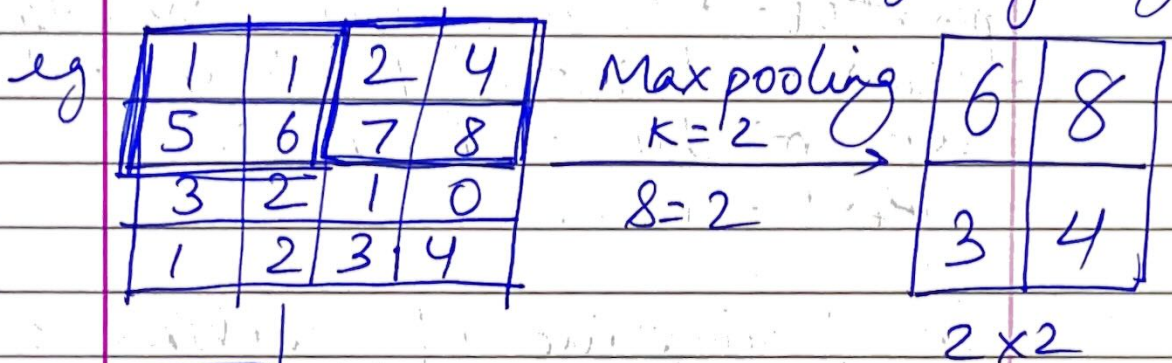
Page No. :

V6: Max pooling

We want our models be location invariant (wherever the object is we should detect it)

Scale-Invariance — For Size.

Rotation — " — Angle of image.



Find max no. in each matrix after applying stride. This is what max pooling is.

max pooling
 $K=2$

$S=1$

6	7	8
6	7	8
3	3	4

3x3

How does max pooling create invariance?

If you have an edge in any cell of the matrix, the cell matrix's highest value is taken

and we say that an edge is present by taking the high value.

V7: CNN Training : Optimization

Convolution layer has convolution operator $\rightarrow$ Elementwise multiplication + addition (it is just like dot product). And it is differentiable.

So derivative of pooling needs to be taken into consideration. — on.

Derivative of max value w.r.t itself is 1.

So derivative of max value = 1
if " " non " " = 0

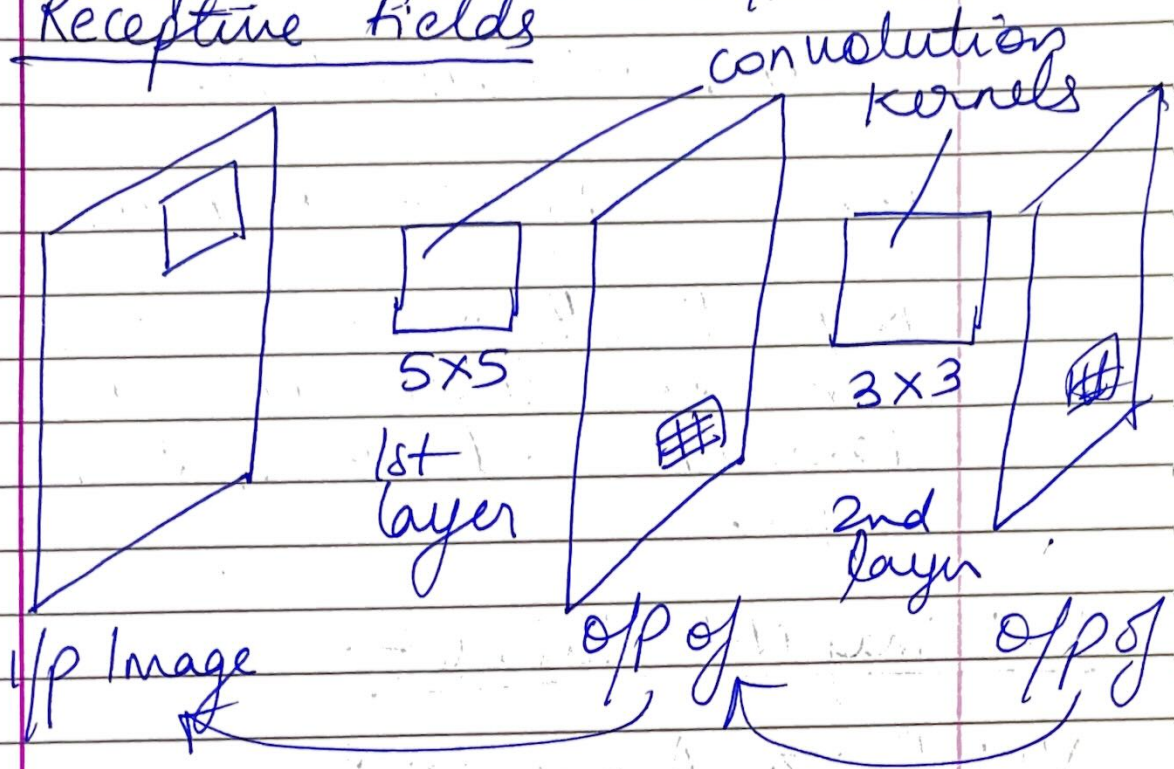
Similar for mean pooling and average pooling.



Date :

Page No. :

V8: Receptive Fields and Effective Receptive Fields



Receptive Field is that part of image which the k convolution kernel operates on at a given point in time.

This operates on I/P image. Different regions have different Receptive fields from that convolution kernels on that I/P image.

O/p gets generated and another kernel is applied on that o/p.

V9: Example CNN: LeNet [1998]

LeCun introduced it.

Feature Maps - kernels.

They didn't have ReLU then so they applied Sigmoid and then softmax layer.

V10: ImageNet dataset

VII: Data Augmentation

Adding something new

We can give instead of 1 image, the augmented copies of that image. Augmentation means doing operations like flip, horizontal shift, vertical shift, rotation, Zoom, etc.

Why it is useful?

- You get invariances
 - * like Shift invariance.
- Rotation invariances
- Zoom "
- Shear "
- Noise "



Date :

Page No. :

You can augment your small dataset to get large dataset esp. for image dataset.

V12: Convolution layers in Keras

Code Walkthrough.

Flatten Converts an n -D data to 1D data.

V13: AlexNet

was applied on ImageNet.

Based on Research paper by Alex K et-al. including Geoff Hinton.

Convolution with same pooling doesn't change $n \times m$ (kernels)

But applying changing it to max pooling changes channels.

Activation used were:-

- 1) ReLU
- Use of dropouts.
- Used multiple GPU's.

New → Local Response Normalization (It normalizes channels instead of the whole Tensor)

V14: VGGNet

Research paper

Books by Andrew Zisserman ~~at~~ ~~the~~ On Computer Vision.

Better version of AlexNet.

In each layer:-

Same Convolution, stride and padding. Also same maxpooling is written in VGGNet unlike different in each layer in AlexNet.

Keras has pre trained VGG16.
Also available in Tensorflow.

V15: Residual Network (ResNet)

Research from Microsoft

Even a 56 layer NN had Training error and Test error were higher than a 20 layer network.

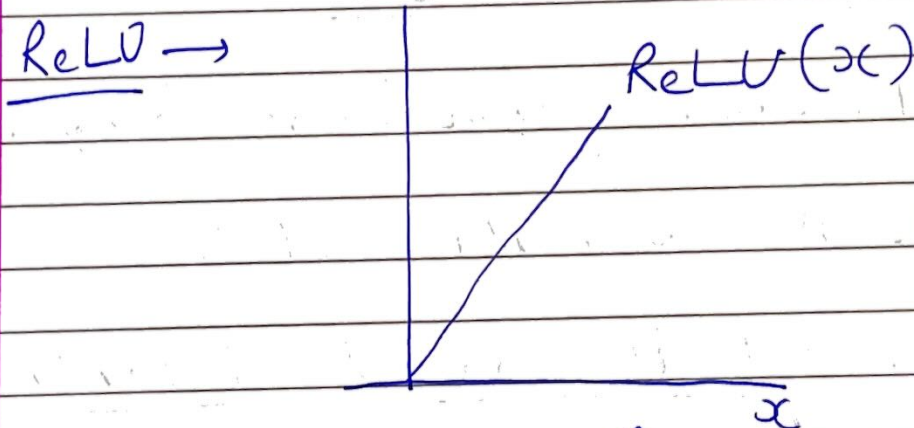
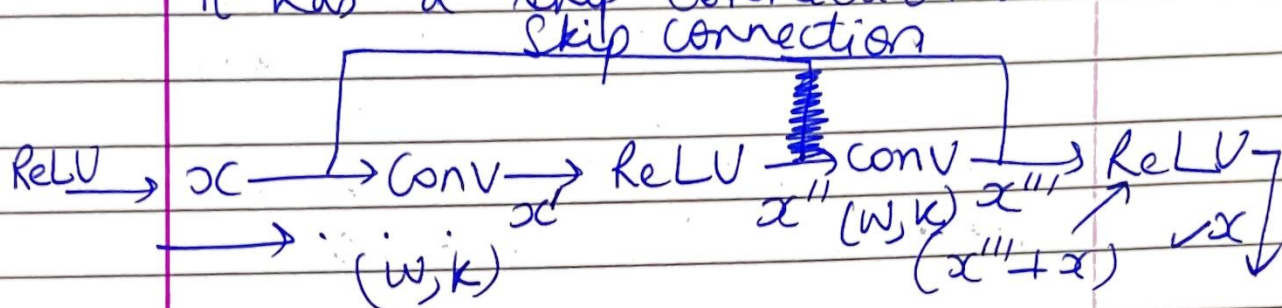
If it is features we perform dot product, if images we perform convolution.



Date :

Page No. :

ResNet has a Identity / Res Block.
It has a skip connection also.



$$\text{ReLU}(\text{ReLU}(x)) = \text{ReLU}(x)$$

$$x = +ve$$

$$\text{ReLU}(x) = x$$

$$\text{ReLU}(\text{ReLU}(x)) = x$$

$$x = -ve$$

$$\text{ReLU}(x) = 0$$

$$\text{ReLU}(\text{ReLU}(x)) = 0 = \text{ReLU}(x)$$

Regularizations remove all the useless weights.

from this formula

$$\text{ReLU}(x_0) = x$$

$$\text{ReLU}(\text{ReLU}(x_0)) = \text{ReLU}(x_0) = x$$

- 1) Adding additional/new layers would not hurt performance as regularization will skip over them.
- 2) If Alex layers are useful ~~as~~ regularization →

V16: Inception Network

Research paper from Google.

Keras has implemented it.

V17: What is Transfer Learning

Instead of building a CNN from scratch, what if we can reuse existing model architecture which is trained on a different dataset.

Keras has VGG16 trained on ImageNet.

Also available on Tensorflow.

Can I transfer my learning from 1 dataset to another.

Hence called Transfer Learning



Date :

Page No. :

Case 1: VGG16

Just before the flatten we have the bottleneck features,

1/p each x_i till before flatten layer, take the vector till here

You get a $x_i \in \mathbb{R}^d$

VGGNet is used as a feature engineering technique and the vector that you get with this is CNN-code.

After that you can use a Logistic Regression technique.

Case 2: VGG16 + ImageNet

We will freeze the initial convolution blocks, i.e., make weights ~~and~~ as constants and just modify the last 2 layers' weights as variable.

Case 3: VGG16 + ImageNet

Use $\uparrow$ as initial model and tune the complete model on your new dataset. But keep low learning rate.

Case 4: Reobtain the network from scratch X Don't do this.

~~Taking~~ .

Using any case as model depends on size of dataset.

GitHub blog for Transfer learning methods under various cases.